

A Comparative Review of Deep Learning Optimizers: Synthesis of Three Empirical Studies

Week 3 – Day 2 Research Task
Training Artificial Neural Networks

July 22, 2026

Abstract

Choosing an optimizer is one of the most consequential, and most under-justified, decisions in training a neural network. This report reviews three recent empirical studies that compare gradient-based optimizers, Soydaner (2020), Almog (2025), and a 2025 comparative study published in the International Journal of Innovative Science and Research Technology (IJISRT), and synthesizes their findings into practical guidance. Across all three studies, Adam-family optimizers (Adam, AdaMax, Nadam, AMSGrad, AdamW) emerge as the most reliable low-tuning-effort default, particularly as dataset size and model depth increase, while a carefully tuned SGD remains competitive, and at times superior, on smaller or simpler problems. The trade-off is not accuracy alone: Almog (2025) shows that this advantage can come at a measurable energy cost. No single optimizer dominates every dataset, architecture, and objective simultaneously.

1 Introduction

Gradient descent and its variants update a network’s parameters θ by moving against the gradient of a cost function $J(\theta)$:

$$\theta \leftarrow \theta - \eta \nabla J(\theta)$$

where η is the learning rate. In practice, plain gradient descent is rarely used directly; instead, optimizers such as Momentum, RMSProp, and Adam modify this update rule to accelerate convergence, dampen oscillation, or adapt the effective learning rate per parameter. Because these behaviours interact with dataset size, architecture depth, and hardware constraints in ways that are difficult to predict analytically, the choice between optimizers is largely an empirical question. This report reviews three papers that address it directly.

2 Paper 1: Soydaner (2020)

2.1 Overview

Soydaner [1] empirically compares ten widely used gradient-based optimizers, SGD, SGD with Momentum, SGD with Nesterov Momentum, AdaGrad, AdaDelta, RMSProp, Adam, AdaMax, Nadam, and AMSGrad, across both supervised (image classification) and unsupervised (autoencoder reconstruction) tasks.

- **Datasets:** MNIST, CIFAR-10, a Kaggle Flowers dataset, and Labeled Faces in the Wild (LFW), evaluated across three convolutional network depths and two autoencoder depths.
- **Optimizers evaluated:** SGD, SGD+Momentum, SGD+Nesterov, AdaGrad, AdaDelta, RMSProp, Adam, AdaMax, Nadam, AMSGrad.

- **Key findings:** Adam and its close variants (AdaMax, Nadam, AMSGrad) consistently outperformed the remaining adaptive methods. Plain SGD’s performance dropped sharply on the harder, higher-resolution LFW dataset, and AdaGrad struggled throughout the benchmark suite. AdaDelta required no manual learning-rate tuning while remaining competitive with the tuned methods.
- **Limitations:** Experiments were run on a single GPU with hyperparameters drawn largely from prior literature rather than tuned individually per optimizer; scope was restricted to image classification and reconstruction.

2.2 Analysis

The central result, that the performance gap between SGD and adaptive methods widens as problems become harder, helps explain why Adam is treated as a common default starting point in practice. At the same time, the paper is careful to show that the ranking between optimizers is not fixed: it shifts noticeably with dataset difficulty and network architecture, which argues against treating any optimizer choice as universally settled.

3 Paper 2: Almog (2025)

3.1 Overview

Almog [2] reframes the optimizer-selection question around a dimension the other two papers do not consider directly: energy consumption. The study quantifies the trade-off between accuracy, training time, and carbon emissions across eight optimizers.

- **Datasets:** MNIST, CIFAR-10, and CIFAR-100, evaluated with 8 optimizers $\times$ 15 random seeds (360 total experiments) on Apple M1 Pro hardware, with emissions tracked using CodeCarbon.
- **Optimizers evaluated:** SGD, Adam, AdamW, RMSprop, Adagrad, Adadelata, Adamax, and NAdam.
- **Key findings:** AdamW and NAdam were consistently efficient choices across datasets. SGD achieved superior accuracy specifically on the more complex CIFAR-100 dataset, but at a significantly higher energy cost. Emissions and training duration were only loosely correlated with each other, meaning faster training did not reliably imply lower emissions.
- **Limitations:** Results are drawn from a single hardware platform (Apple M1 Pro), fixed literature-standard hyperparameters, and a scope limited to relatively small image-classification CNNs.

3.2 Analysis

This study is valuable precisely because it introduces a cost axis that pure-accuracy comparisons omit. Its practical implication is a two-tier heuristic: favour energy-efficient optimizers such as AdamW or NAdam by default on easier tasks, and reserve SGD’s higher energy cost for the harder problems where it demonstrably earns its accuracy advantage.

4 Paper 3: IJISRT Comparative Study (2025)

4.1 Overview

This study [3] benchmarks seven optimizers, including two newer methods, Yogi and Lion, on training and test accuracy, training loss, and sensitivity to learning-rate choice.

- **Datasets:** MNIST and CIFAR-10.

- **Optimizers evaluated:** SGD, Momentum SGD, RMSProp, AMSGrad, Adam, Yogi, and Lion.
- **Key findings:** SGD with a learning rate of 0.5 achieved the best test accuracy (99.14%) and the highest training accuracy (99.89%) on MNIST. Momentum SGD and Adam also performed strongly, reaching 99.15% and 98% test accuracy respectively at a learning rate of 10^{-2} . The paper’s related-work summary additionally notes Adam reaching top test accuracy in other studies (98.26% on MNIST, 98.55% on CIFAR-10), with RMSProp excelling specifically on training accuracy.
- **Limitations:** Restricted to two comparatively easy benchmark datasets; the newer Yogi and Lion optimizers underperformed the classic methods in this particular setting.

4.2 Analysis

This result reinforces a theme visible across all three papers: a well-tuned SGD can match or exceed adaptive optimizers on simpler benchmarks, even though Adam-family methods converge faster and are more forgiving of learning-rate choice. It is a concrete instance of the “no free lunch” principle applied to optimizer selection, performance on an easy, low-noise dataset does not necessarily predict performance on a harder one.

5 Cross-Paper Synthesis

Table 1 summarizes the three studies. Read together, they converge on a consistent, practical conclusion:

- **Adam-family optimizers** (Adam, AdamW, Nadam) are the safest, lowest-tuning-effort default, and this advantage grows as dataset size and model complexity increase (Soydaner, 2020).
- **Well-tuned SGD remains competitive**, and sometimes superior, on simpler, well-behaved problems, at the cost of more careful learning-rate tuning (IJISRT, 2025) and, per Almog (2025), higher energy consumption on complex tasks.
- **No optimizer dominates universally.** The ranking between methods shifts with dataset difficulty, model depth, and the metric being optimized for (accuracy vs. training time vs. emissions), so optimizer choice should be treated as a tunable hyperparameter rather than a fixed default.

Paper	Datasets	Headline finding
Soydaner (2020)	MNIST, CIFAR-10, Flowers, LFW	Adam-family methods consistently beat plain SGD/AdaGrad as task difficulty rises.
Almog (2025)	MNIST, CIFAR-10, CIFAR-100	AdamW/NAdam are most energy-efficient; SGD wins on accuracy for hard tasks but at higher emissions.
IJISRT (2025)	MNIST, CIFAR-10	A well-tuned SGD ($\text{lr} = 0.5$) can match or beat Adam-family optimizers on simpler benchmarks.

Table 1: Summary of the three reviewed optimizer-comparison studies.

6 Conclusion

The practical takeaway for training an ANN, consistent with all three reviewed studies, is to start with Adam or AdamW as a default optimizer, since it is comparatively insensitive to learning-rate choice and performs reliably across dataset scales. Where compute budget or energy cost is a binding constraint on a simpler, well-behaved dataset, a carefully tuned SGD (with an appropriate learning-rate schedule) is a reasonable,

and sometimes superior, alternative. The right choice ultimately depends on dataset difficulty, model depth, and which objective, accuracy, training time, or energy cost, is being optimized for.

References

- [1] Soydaner, D. (2020). *A Comparison of Optimization Algorithms for Deep Learning*. International Journal of Pattern Recognition and Artificial Intelligence. arXiv:2007.14166.
- [2] Almog, T. (2025). *An Analysis of Optimizer Choice on Energy Efficiency and Performance in Neural Network Training*. University of Waterloo. arXiv:2509.13516.
- [3] *Comparative Study of Deep Learning Optimizers: SGD, Momentum SGD, RMSProp, AMSGrad, Adam, Yogi, and Lion* (2025). International Journal of Innovative Science and Research Technology, 10(8).
- [4] Kingma, D. P., & Ba, J. (2014). *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980.
- [5] Ruder, S. (2016). *An Overview of Gradient Descent Optimization Algorithms*. arXiv:1609.04747.
- [6] Loshchilov, I., & Hutter, F. (2017). *Decoupled Weight Decay Regularization*. arXiv:1711.05101.
- [7] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [8] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.